

Asymptotic Notations

Big-Oh, Omega, and Theta

Ordering of some important functions as $n \rightarrow \infty$

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms

Basic asymptotic efficiency classes

1	constant
$\log n$	logarithmic
n	linear
$n \log n$	n-log-n
n^2	quadratic
n^3	cubic
2^n	exponential
$n!$	factorial

Math you need to Review

- **Summations**
- **Logarithms and Exponents**
 - properties of logarithms:
 - $\log_b(xy) = \log_b x + \log_b y$
 - $\log_b (x/y) = \log_b x - \log_b y$
 - $\log_b x^a = a \log_b x$
 - $\log_b a = \log_x a / \log_x b$
 - properties of exponentials:
 - $a^{(b+c)} = a^b a^c$
 - $a^{bc} = (a^b)^c$
 - $a^b / a^c = a^{(b-c)}$
 - $b = a^{\log_a b}$
 - $b^c = a^{c \cdot \log_a b}$
- **Proof techniques**
- **Basic probability**

Big-Oh, Omega, and Theta

- Big-Oh, Omega, and Theta, are the most common asymptotic notations. (Small-oh and omega are less common.)
- In most cases time-complexity of algorithms are described by these notations, but **the notations themselves are basically used to describe functions.**
- In other words, by definitions, the notations are originally defined for functions, **NOT** for time-complexity.

Definitions of Asymptotic Notations

A way of comparing functions that ignores constant factors and small values.

Below, n_0 is positive integer, c, c_1, c_2 are positive real numbers. But, sometimes, they can be integers. The dot $\bullet$ means “such that”.

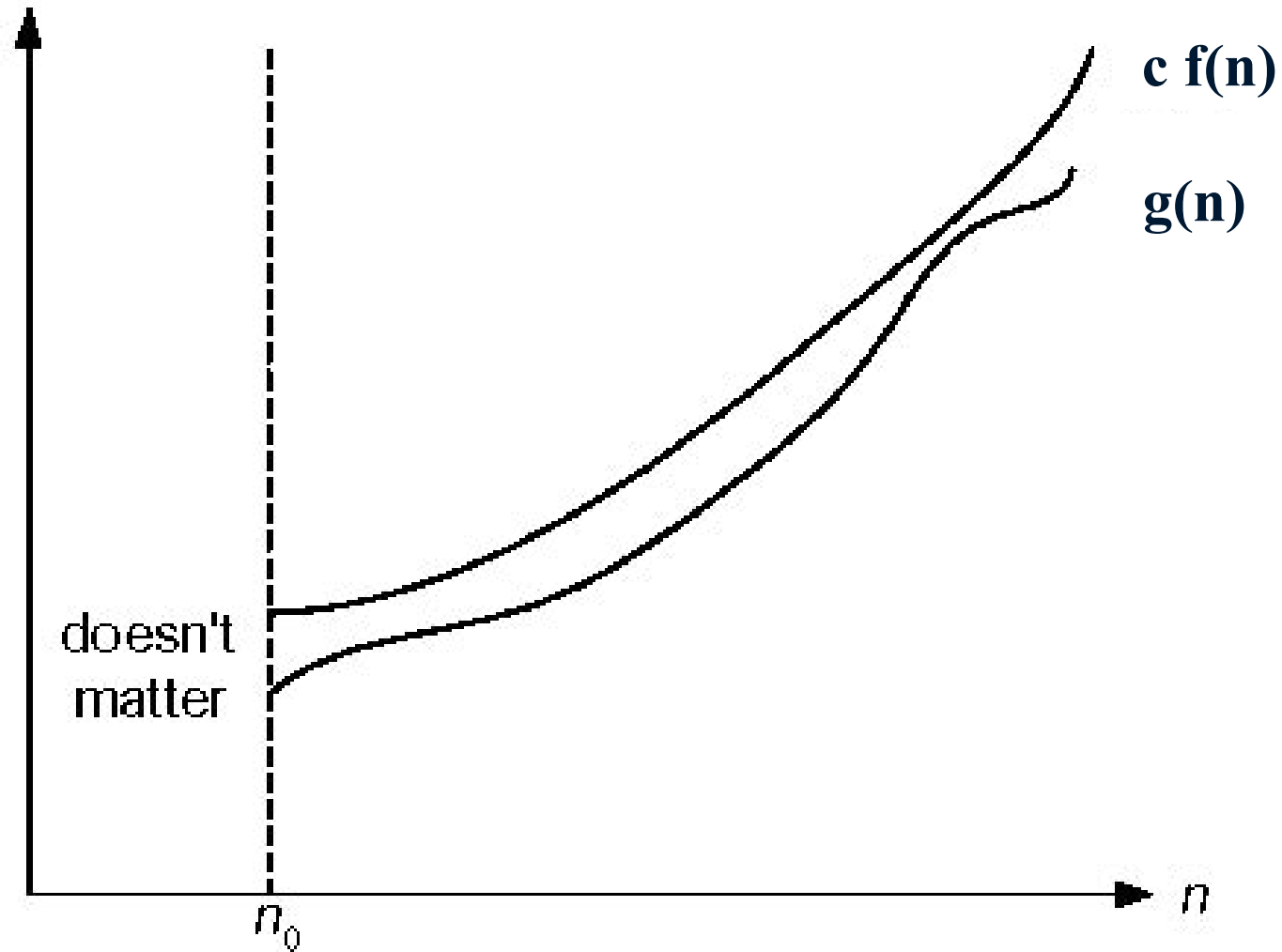
Big-O : $O(f(n)) = \{g(n) \mid \exists c, n_0 \forall n > n_0 \bullet g(n) \leq c f(n)\}$

Big-Ω : $\Omega(f(n)) = \{g(n) \mid \exists c, n_0 \forall n > n_0 \bullet g(n) \geq c f(n)\}$

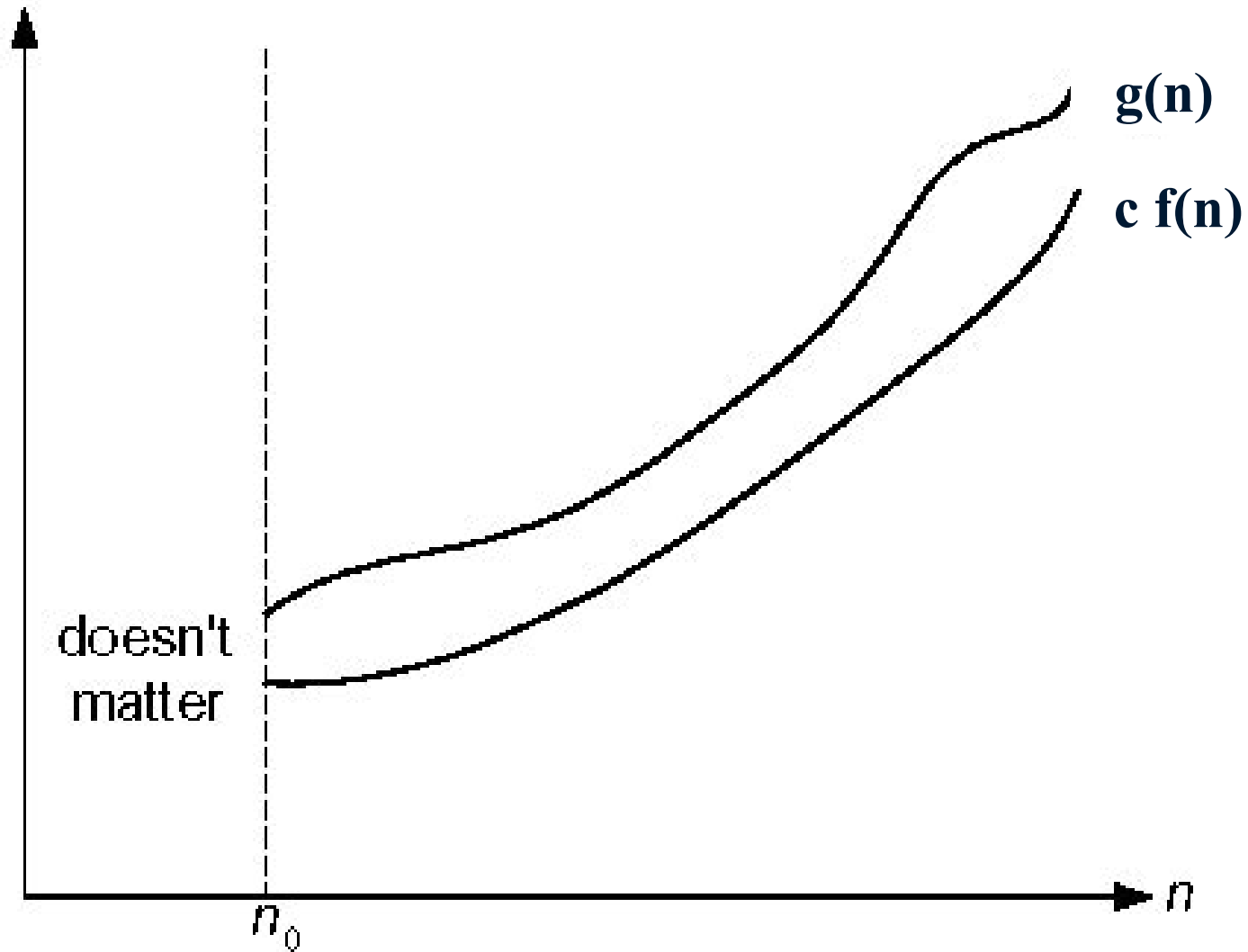
Big-Θ : $\Theta(f(n)) = \{g(n) \mid \exists c_1, c_2, n_0 \forall n > n_0 \bullet c_2 f(n) \leq g(n) \leq c_1 f(n)\}$

The most common one is Big-O (also known as Big-Oh).

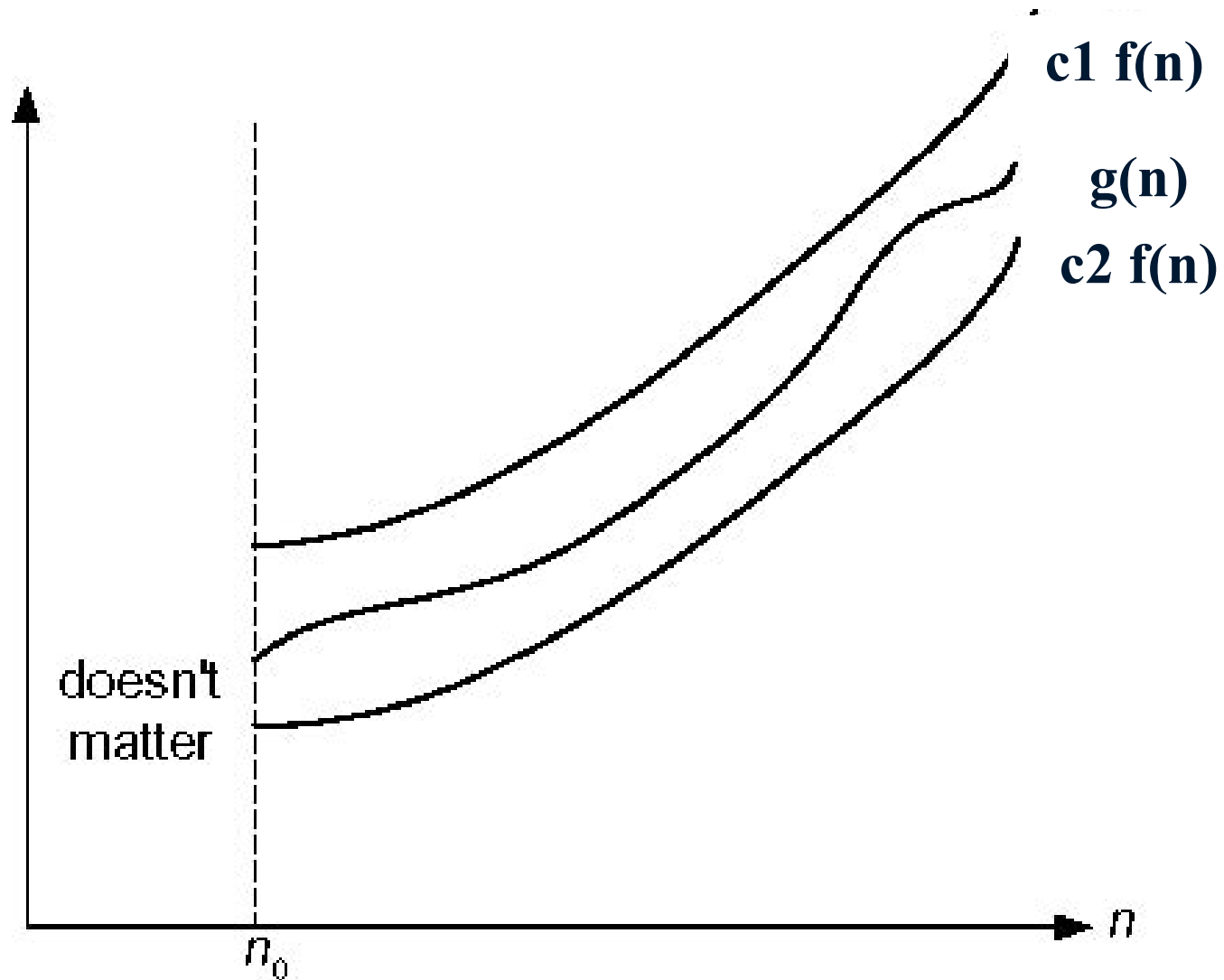
Graphical meaning of $g(n) = O(f(n))$



Idea of $g(n) = \Omega(f(n))$



Big-theta, $g(n)=\Theta(f(n))$



Use of Big-Oh, Omega, and Theta

- Definitions are complicate but the notations themselves are easy to use.
- Example:
- $n = O(n)$; $100n+5 = O(n)$; $\frac{1}{2} n(n-1) = \frac{1}{2} n^2 - \frac{1}{2} n = O(n^2)$
- $n^3 + 20n^2 + 2n + 109 = O(n^3)$; $0.00001n^3 + \frac{1}{2} n^2 = O(n^3)$
- $n^4 + n + 1 = O(n^4)$

- $n = \Omega(n)$; $100n+5 = \Omega(n)$; $\frac{1}{2} n(n-1) = \frac{1}{2} n^2 - \frac{1}{2} n = \Omega(n^2)$
- $n^3 + 20n^2 + 2n + 109 = \Omega(n^3)$; $0.00001n^3 + \frac{1}{2} n^2 = \Omega(n^3)$
- $n^4 + n + 1 = \Omega(n^4)$

- $n = \Theta(n)$; $100n+5 = \Theta(n)$; $\frac{1}{2} n(n-1) = \frac{1}{2} n^2 - \frac{1}{2} n = \Theta(n^2)$
- $n^3 + 20n^2 + 2n + 109 = \Theta(n^3)$; $0.00001n^3 + \frac{1}{2} n^2 = \Theta(n^3)$
- $n^4 + n + 1 = \Theta(n^4)$

- The only trick for all Big-Oh, Ω , and Θ :
 - Remove the lower order terms, remove the coefficient.
- Note, these asymptotic notations are sets, and “=” means “ $\in$ ”.
- Traditionally, we use “=” rather than “ $\in$ ”.

More examples on Big-Oh, Omega, and Theta

- If $10n^2 + 2n + (\log n)^{10} < f(n) < n^3 + 20n^2 + (\log n)^{10}$
- then $f(n) = O(n^3)$, and $f(n) = \Omega(n^2)$.

- If $n^3 + 300n^2 + 200n + 109 < f(n) < 3n^3 + 20n^2 + 2n + 109$
- then $f(n) = O(n^3)$, and $f(n) = \Omega(n^3)$.
- then $f(n) = \Theta(n^3)$.

More examples on Big-Oh, Omega, and Theta

- If $f(n) < 3n^3 + 20n^2 + 2n + 109 < 10n^4$
 - then $f(n) < 10n^4$
 - then $f(n) = O(n^4)$
 - Correct? Yes, but not tight.
-
- If $f(n) > n^3 + 300n^2 + 200n + 109$
 - Then $f(n) > n^2$
 - Then $f(n) = \Omega(n^2)$
 - Correct? Yes, but not tight.
-
- We should use the tightest format.

Why tightest?

- Imagine a policeman caught a suspect, and asked him how much money he has stolen.
- The suspect replied: less than one thousand RM.
- Then, the policeman searched the suspect, and found only 10RM in his pocket.
- He asked: why did you lie to me? You have only 10RM, and how could you say 1000?
- The suspect replied: I did tell the truth, because $10 < 1000$.
- ...
- ...
- Students, imagine what will happen to the suspect.

Big-Oh Practices

- If $f(n)$ is a polynomial of degree d , then $f(n)$ is $O(n^d)$, i.e.,
 1. Drop lower-order terms
 2. Drop constant factors
- Use the smallest possible class of functions
 - Say “ $2n = O(n)$ ” instead of “ $2n = O(n^2)$ ”
 - Both are correct, but the former is better.
- Use the simplest expression of the class
 - Say “ $3n + 5 = O(n)$ ” instead of “ $3n + 5 = O(3n)$ ”
 - Both are correct, but the first one is cleaner.
 - Indeed, $O(n) = O(3n) = O(3n+5)$, according to the definition.

Asymptotic order of growth

Asymptotic: A way of comparing functions that ignores constant factors and small input sizes

- $O(g(n))$: class of functions $f(n)$ that grows no faster than $g(n)$
- $\Theta(g(n))$: class of functions $f(n)$ that grows at same rate as $g(n)$
- $\Omega(g(n))$: class of functions $f(n)$ that grows at least as fast as $g(n)$

Another why.

- Recall:
- Big-O : $O(f(n)) = \{g(n) | \exists c, n_0 \forall n > n_0 \bullet g(n) \leq c f(n)\}$
- Why $0.00001n^3 + \frac{1}{2}n^2 = O(n^3)$?
- In other words, why **removing the lower order terms, and removing the coefficient** is correct?
- Can we prove $0.00001n^3 + \frac{1}{2}n^2 = O(n^3)$ by the definition of big-O? Yes.
- Since $\exists c = 1, n_0 = 1 \bullet \forall n > n_0 \bullet 0.00001n^3 + \frac{1}{2}n^2 \leq c n^3$, we can say $0.00001n^3 + \frac{1}{2}n^2 = O(n^3)$.
- Same to Ω and Θ .

Prove $200n^2 + 7n = O(n^2)$

- Rough work
- $200n^2 + 7n \leq cn^2$
- $\Leftarrow$ if $c=207, n>1$
- -----
- Your work:
- Take $c=207, n_0=1$.
- Then, for all $n>n_0, 200n^2 + 7n \leq cn^2$
- So, $200n^2 + 7n = O(n^2)$.

Alternative answer for $200n^2 + 7n = O(n^2)$

- Rough work
- $200n^2 + 7n \leq cn^2$
- Set $c=201$, $200n^2 + 7n \leq 201n^2$
- So, $7n \leq n^2$. So, $n \geq 7$.
- -----
- Your work:
- Take $c=201$, $n_0=7$.
- Then, for all $n > n_0$, $200n^2 + 7n \leq cn^2$
- So, $200n^2 + 7n = O(n^2)$.

Prove $3\log n + 100 = O(\log n)$

- Take $c=4$, $n_0=2^{100}$.
- For all $n > n_0$
- $n \geq 2^{100}$
- $\log n \geq 100$
- $3 \log n + 100 \leq 4 \log n$
- $3 \log n + 100 \leq c \log n$
- So, $3\log n + 100 = O(\log n)$.

Prove $3\log n + 100 = O(\log n)$

- Take $c=103$, $n_0= 1$.
- For all $n > n_0$
- $103 \log n \leq c \log n$
- $3 \log n + 100 \log n \leq c \log n$
- $3 \log n + 100 \leq c \log n$
- So, $3\log n + 100 = O(\log n)$.

Proof for $\neq$, begin with logic

- Example: $n = O(n^2)$, but $n^3 \neq O(n^2)$.
- Traditionally, in this context, “ $\notin$ ” is written as “ $\neq$ ”.
- Recall $O(f(n)) = \{g(n) \mid \exists c, n_0 \forall n > n_0 \bullet g(n) \leq c f(n)\}$.
- To prove $n^3 \neq O(n^2)$
- $\equiv$ To prove $[\exists c, n_0 \forall n > n_0 \bullet n^3 \leq c n^2]$ is wrong
- $\equiv$ To prove $\neg[\exists c, n_0 \forall n > n_0 \bullet n^3 \leq c n^2]$
- $\equiv$ To prove $\forall c \neg[\exists n_0 \forall n > n_0 \bullet n^3 \leq c n^2]$
- $\equiv$ To prove $\forall c \forall n_0 \neg[\forall n > n_0 \bullet n^3 \leq c n^2]$
- $\equiv$ To prove $\forall c \forall n_0 \exists n > n_0 \bullet \neg[n^3 \leq c n^2]$
- $\equiv$ To prove $\forall c \forall n_0 \exists n > n_0 \bullet n^3 > c n^2$.

- Intuitively, the reason for $n^3 \neq O(n^2)$ is $n^3 > c n^2$ is true: No matter what values of c and n_0 are given, we can still find an $n > n_0$ such that $n^3 > c n^2$ is true.

Prove $n^3 \neq O(n^2)$.

- We need to prove
- $[\exists c, n_0 \bullet \forall n > n_0 \bullet n^3 \leq cn^2]$ is false.
- It is equivalent to prove
- $[\forall c, n_0 \bullet \exists n > n_0 \bullet n^3 > cn^2]$ is true.
- Proof: $\forall c, n_0 \bullet \exists n = \max(n_0, \lceil c \rceil) + 1 > n_0 \bullet n^3 > cn^2$
- Done.
- In more natural language, we can say:
- Given any ($\forall$) c and n_0 , take ($\exists$) $n = \max(n_0, \lceil c \rceil) + 1 > n_0$.
- Then, $n^3 > cn^2$. Done.

Same to Ω and Θ

■ **Big- Ω :** $\Omega(f(n)) = \{g(n) \mid \exists c, n_0 \forall n > n_0 \bullet g(n) \geq c f(n)\}$

■ **Example:** (Remember = means $\in$, $\neq$ means $\notin$.)

■ $n^3 \in \Omega(n^2);$ $\frac{1}{2} n(n-1) \in \Omega(n^2)$

■ $100n + 5 \notin \Omega(n^2)$

■ **Big- Θ :** $\Theta(f(n)) = \{g(n) \mid \exists c_1, c_2, n_0 \forall n > n_0 \bullet c_2 f(n) \leq g(n) \leq c_1 f(n)\}$

■ Informally, $\Theta(g(n))$ is the set of all functions that have the same order of growth as $g(n)$ (to within a constant multiple, as n goes to infinity). Example, every quadratic function $an^2 + bn + c$ for $a > 0$ is $\Theta(n^2)$

Prove $\frac{1}{2} n^3 = \Omega(n^2)$

- Rough work
- $(\frac{1}{2})n^3 \geq cn^2$
- $(\frac{1}{2})n \geq c$
- $n \geq 2c$
- Set $c=1$, $n_0=2$.
- -----
- Your work:
- Take $c=1$, $n_0=2$.
- Then, for all $n > n_0$, $n^3 \geq cn^2$
- So, $n^3 = \Omega(n^2)$.

Prove $100n + 5 \neq \Omega(n^2)$

- We need to prove
- $[\exists c, n_0 \bullet \forall n > n_0 \bullet 100n + 5 \geq cn^2]$ is false.
- It is equivalent to prove
- $[\forall c, n_0 \bullet \exists n > n_0 \bullet 100n + 5 < cn^2]$ is true.
- Proof: $\forall c, n_0 \bullet \exists n = \max(n_0, \lceil 101/c \rceil) + 5 > n_0 \bullet 100n + 5 < cn^2$
- Done.
- In more natural language, we can say:
- Given any $(\forall) c$ and n_0 , take $(\exists) n = \max(n_0, \lceil 101/c \rceil) + 5 > n_0$.
- Then, $100n + 5 < cn^2$. Done.

Some properties of the notations

- $f(n) = O(f(n))$
- $f(n) = O(g(n))$ iff $g(n) = \Omega(f(n))$
- If $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$
- If $f_1(n) = O(g_1(n))$ and $f_2(n) = O(g_2(n))$,
- then $f_1(n) + f_2(n) = O(\max\{g_1(n), g_2(n)\})$
- $[f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))]$ iff $f(n) = \Theta(g(n))$
 - We can prove $f(n) = \Theta(g(n))$ by proving BOTH $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
 - We can prove $f(n) \neq \Theta(g(n))$ by proving $f(n) \neq O(g(n))$ or $f(n) \neq \Omega(g(n))$.

Four equivalent definitions for $O(f(n))$

- $\{g(n) | \exists c, n_0 \forall n \geq n_0 \bullet g(n) \leq c f(n)\}$
- $\{g(n) | \exists c, n_0 \forall n > n_0 \bullet g(n) < c f(n)\}$
- $\{g(n) | \exists c, n_0 \forall n \geq n_0 \bullet g(n) < c f(n)\}$
- $\{g(n) | \exists c, n_0 \forall n > n_0 \bullet g(n) \leq c f(n)\}$

■ Are they really identical? YES.

■ Can be proven? YES.

■ Time to sharpen our sense of infinity₁₋₂₇

Exercise

- Let $A = \{g(n) | \exists c, n_0 \forall n \geq n_0 \bullet g(n) \leq c f(n)\}$, and
- $B = \{g(n) | \exists c, n_0 \forall n \geq n_0 \bullet g(n) < c f(n)\}$
- Prove $A=B$.

Time Complexity

Worst Case, Average Case, Best Case

Definition of Time Complexity

- Time Complexity is the execution time needed for an algorithm for large input length, regardless of the language used and execution platform.
- However, time needed depends on the input content. So, there are three types of time complexities:
 - **Worst case** time complexity
 - **Average case** time complexity
 - **Best case** time complexity
- Normally, **time complexity is referred to worst case time complexity;** if not specified otherwise.
- Note, all the time complexities are for large input length, regardless of the language used and execution platform.

A misconception about average case:

- Average time
- $= (\text{best case time} + \text{worst case time}) / 2$
- For quicksort,
 - best case time complexity $= O(n \log n)$
 - average case time complexity $= O(n \log n)$
 - worst case time complexity $= O(n^2)$
- For a random search in an array
 - best case time complexity $= O(1)$
 - average case time complexity $= O(n)$
 - worst case time complexity $= \text{infinity}$



Theoretical analysis of time efficiency

**Time complexity is the total number of primitive operations.
However, normally we need NOT consider all kinds of operations. Why?**

Example: For i = 1 to n

$A[i] = A[i] * 2;$

- Basic operations: * and = .
- Normally, the operations for loop counter need not be counted.
- For *, n times; for =, n times.
- We can choose any one operation. The same result $O(n)$ will follow.
- Normally, we choose the most costly operation. Here, it is *.

Theoretical analysis of time efficiency

Example: **For i = 1 to n**
 For j = 1 to n
 if i = j
 x = x*2;

- Basic operations:
- *, =, and comparison (subtraction?)
- For both * and =, n times; for comparison, n^2 times.
- Which operation is used for counting time complexity?
- If the cost of a multiplication is n times the cost of a comparison, we should count multiplication.
- But if we consider multiplication is the one on our ALU, constant time, then we consider comparison. $\rightarrow O(n^2)$ time.

Theoretical analysis of time efficiency

- Uses a high-level description of the algorithm instead of an implementation
- Characterizes running time as a function of the input size, n .
 - ⌚ Sometimes, as a function of the input value.
 - ⌚ Beware of this difference.
- Takes into account all possible inputs
- → Mathematical proof is not avoidable.
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment
- → Order of Growth is used

Example: Sequential search (1)

ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element of A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

- **Best case:** Best case input of size n for which the algorithm runs fastest. In this example, best case happens when key K is found in first element of the array.
- **Worst case:** When the key K is found in the last element of the array, or even not found in the array at all after comparing all the elements in the array; $i=n$

Example: Sequential search (2)

ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element of A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

■ **Average case:**

Input size and basic operation examples

Problem	Input size measure	Basic operation
Searching for key in a list of n items	Number of list's items, i.e. n	Key comparison
Multiplication of two matrices	Matrix dimensions or total number of elements	Multiplication of two numbers
Checking primality of a given integer n	n' size = number of digits (in binary representation) $\text{floor}(\log n) + 1$ bits	Division
Typical graph problem	#vertices and/or edges and	Visiting a vertex or traversing an edge

Prove that the length of n is $\text{floor}(\log n)+1$.

- **Statement:** For any length-k binary string beginning with “1”, the minimum number is 2^{k-1} .
- **Proof:** Use induction on k.
- **Base case:** $k=1$.
- The length-1 binary string starting with “1” is $1=2^{1-1}$.
- Hence, the statement is true for the base case.

- Assume that the statement is true for $k=m \geq 1$.
- The minimum number of a length-m binary string is $100\dots 0$, with m-1 zeroes.
- Consider the case for length= $m+1$. The minimum number of a length-($m+1$) binary string is $100\dots 0$, with m zeroes.
- This number is $2 \times (\text{the minimum number of a length-m binary string})$,
- and this equals to $2 \times (2^{m-1}) = 2^m$.
- Hence, the statement is also true for $k=m+1$.
- By the principal of mathematical induction, the statement is true for all all values of $k \geq 1$.

Prove that the length of n is $\text{floor}(\log n)+1$.

- **Statement:** For any length-k binary string beginning with “1”, the minimum number is 2^{k-1} .
- **Proof:** Use induction on k.
- **Base case:** $k=1$.
- The length-1 binary string starting with “1” is $1=2^{1-1}$.
- Hence, the statement is true for the base case.

- Assume that the statement is true for $k=m \geq 1$.
- The minimum number of a length-m binary string is $100\dots 0$, with m-1 zeroes.
- Consider the case for length= $m+1$. The minimum number of a length-($m+1$) binary string is $100\dots 0$, with m zeroes.
- This number is $2 \times (\text{the minimum number of a length-m binary string})$,
- and this equals to $2 \times (2^{m-1}) = 2^m$.
- Hence, the statement is also true for $k=m+1$.
- **Result from induction.**

Prove that the length of n is $\text{floor}(\log n)+1$.

- We just prove that for any length-k binary string beginning with “1”, the minimum number is 2^{k-1} .
- By similar method, for any length-k binary string beginning with “1”, the maximum number is 2^k-1 . (Try this by yourself)
- Grouping two statements together, for any length-k binary number n beginning with “1”, n is between 2^{k-1} and 2^k-1 .
- Then, $2^{k-1} \leq n \leq 2^k-1$.
- $\log(n+1) \leq k \leq \log(n) + 1$
- $\log(n) < \log(n+1) \leq k \leq \log(n) + 1$
- Since k is an integer, we have $\lceil \log(n) \rceil \leq \lceil \log(n+1) \rceil \leq k \leq \lfloor \log(n) \rfloor + 1$.
- If n is a power of 2, $\lceil \log(n+1) \rceil = \log(n)+1 = \lfloor \log(n) \rfloor + 1$.
- Then, $k = \lfloor \log(n) \rfloor + 1$.
- If n is not a power of 2, then $\log(n)$ is not an integer, then we have $\lceil \log(n) \rceil = \lfloor \log(n) \rfloor + 1$.
- Since $\lceil \log(n) \rceil \leq \lceil \log(n+1) \rceil \leq k \leq \lfloor \log(n) \rfloor + 1$, then we have $\lceil \log(n) \rceil = \lceil \log(n+1) \rceil = k = \lfloor \log(n) \rfloor + 1$.
- Done.